AMRITA VISHWA VIDYAPEETHAM

School of Artificial Intelligence & Computing · Department of AI

ResiKeep: Cloud-Integrated Enterprise Relational Database for Multi-Tenant Student Housing & Facility Operations

Course Code: 23AIE303 — Database Management Systems (DBMS)
Comprehensive Laboratory Evaluation & Technical Report

Team Member	Roll Number	Assigned Contribution Matrix
R Nethra	CB.SC.U4AIE24147	Database Architecture, PostgreSQL DDL Schema, PL/pgSQL Triggers, Foreign Key Constraints, Cloud Deployment Pipeline
Sahaana Shri S K	CB.SC.U4AIE24149	Relational Normalization (1NF–BCNF), Transactional Stored Procedures, FastAPI REST API Endpoints, Frontend Integration

Academic Batch & Group:	Batch B12 · Team 12
Official GitHub Repository:	https://github.com/Nethra-R368/dbms
Database Engine:	PostgreSQL 18 (Relational Engine) with PL/pgSQL Procedural Extensions
Application Stack:	FastAPI (Python 3.12+), SQLAlchemy 2.0 ORM, React 19, Tailwind CSS, Vite
Hosting & Deployment:	Local Windows Service, Single-Server Unified SPA, Live HTTPS Public Tunnel
Evaluation Standard:	23AIE303 DBMS Laboratory Assessment (Project Presentation, Report & Viva)

3. Introduction and Problem Statement

3.1. Project Context & Background

University hostel accommodation represents a high-density, multi-stakeholder operational ecosystem. Modern campus residential management encompasses diverse mission-critical workflows: residential bed and room inventory management, financial fee invoicing and receipt reconciliation, maintenance grievance ticketing, nightly biometric curfew tracking, student out-pass approvals, catering feedback aggregation, and institutional asset logistics. Historically, educational institutions have operated these subsystems through fragmented paper logbooks and disjointed spreadsheet files. These obsolete workflows fail to maintain data integrity, lack auditability, and introduce severe administrative delays.

3.2. Problem Statement

Manual and naive software solutions suffer from four fundamental systemic deficiencies:

- **Concurrency Race Conditions & Double-Bookings:** In the absence of transactional isolation, multiple wardens allocating rooms simultaneously can assign the same bed to different students, causing severe accommodation conflicts.
- **Financial Ledger Inconsistencies:** Fragmented spreadsheets obscure partial payments, miscalculate late penalty fees, and prevent real-time reconciliation against university accounts ledgers.
- **Severe Data Redundancy & Update Anomalies:** Student demographics replicated across separate registers lead to conflicting, stale, or orphaned records upon student status updates or departures.
- **Accountability & Audit Gaps:** Maintenance complaints, emergency out-passes, and campus visitor logs lack immutable server timestamping, leaving no verifiable audit trail.

3.3. System Objectives & Proposed Solution

ResiKeep / SmartHostel resolves these systemic vulnerabilities by implementing an enterprise-grade **PostgreSQL 18** relational database management system. Rather than delegating business rules exclusively to application code, ResiKeep enforces invariants directly within the relational engine using:

- **Active Database Triggers:** PL/pgSQL triggers automatically maintain room occupancy counts and dynamically update student invoice statuses upon receipt of payments.
- **ACID-Compliant Stored Procedures:** Atomic stored functions (allocate_room, vacate_room) isolate room assignment operations, guaranteeing zero double-bookings.
- **Strict Domain & Referential Constraints:** Cascading rules (ON DELETE CASCADE, RESTRICT) and table check constraints preserve database invariants at all times.
- **Role-Based Access Control (RBAC):** Cryptographically secured JWT authentication segregates administrative, warden, student, and accountant privileges.

4. Conceptual and Logical Database Design

4.1. Conceptual Entity Modeling across 6 Business Domains

The conceptual architecture partitions the university housing ecosystem into 19 distinct normalized entities spanning six interconnected functional domains:

1. **Identity & RBAC Domain:** USERS anchors authentication, linking in a 1:0..1 relationship to STUDENT and WARDEN profiles.
2. **Hostel & Facility Domain:** HOSTEL blocks hold a 1:N hierarchy of ROOM units, supervised by assigned WARDEN officers.
3. **Tenure & Allocation Domain:** ROOM_ALLOCATION tracks student residency with state transitions (ACTIVE, COMPLETED), while ROOM_CHANGE_REQUEST manages internal transfers.
4. **Financial Ledger Domain:** Master rate cards in FEE_STRUCTURE instantiate individualized student balances in STUDENT_FEE, settled via audited PAYMENT transactions.
5. **Student Lifecycle Domain:** Encompasses COMPLAINT ticketing, VISITOR gate passes, LEAVE_REQUEST out-passes, and daily ATTENDANCE logs.
6. **Catering & Inventory Domain:** INVENTORY tracks equipment deltas in INVENTORY_TRANSACTION, while MESS_MENU pairs with student ratings in MESS_FEEDBACK.

4.2. Referential Integrity & Cascading Strategies

• **ON DELETE CASCADE:** When a student is removed, dependent allocations, fees, attendance rolls, and complaints are purged automatically to eliminate orphaned records.

• **ON DELETE SET NULL:** If a warden profile is archived, historical complaint logs and hostel supervision records remain intact with warden references set to NULL.

• **ON DELETE RESTRICT:** Tariff tiers in FEE_STRUCTURE cannot be deleted while active student fee records reference them, preventing pricing history corruption.

5. Normalization and Functional Dependencies

To guarantee that the relational schema is completely immune to insertion, update, and deletion anomalies, the database was systematically decomposed from an Unnormalized Form (UNF) through to Boyce-Codd Normal Form (BCNF).

Normal Form	Violation in Naive Schema	Mathematical Decomposition & Resolution
1NF (First Normal Form)	Repeating groups and comma-delimited strings for fee payments, complaints, and roommates within student rows.	Enforced atomic scalar domains. Decomposed multi-valued fields into distinct tables: STUDENT, STUDENT_FEE, PAYMENT, and COMPLAINT with surrogate primary keys.
2NF (Second Normal Form)	Partial functional dependencies on composite candidate keys: In {Student_ID, Room_ID} -> Hostel_Name, Capacity, room attributes depend only on Room_ID.	Eliminated partial dependencies by separating room catalog attributes into ROOM(Room_ID, Hostel_ID, Room_Number, Capacity) and maintaining allocation state in ROOM_ALLOCATION.
3NF (Third Normal Form)	Transitive functional dependencies: In student billing, Fee_ID -> Room_Type -> (Academic_Fee, Mess_Fee) where non-key charges depend transitively on Room_Type.	Eliminated transitive dependencies by isolating master room tariffs into FEE_STRUCTURE(Fee_Structure_ID, Room_Type, Academic_Fee, Mess_Fee), linked via Foreign Key.
BCNF (Boyce-Codd Normal Form)	Occurs when a functional determinant X in non-trivial X -> Y is not a candidate superkey.	Every functional determinant across all 19 relations (e.g. User_ID, Roll_No, {Hostel_ID, Room_Number}, Transaction_ID) is a candidate superkey. Hence, BCNF is strictly satisfied.

6. Advanced SQL Implementation and Query Design

6.1. Analytical Queries: Aggregations, Joins & Window Functions

Below are representative queries demonstrating multi-table relational joins, conditional aggregations, and analytical window functions implemented in the production system:

```
-- Query 1: Hostel-Level Real-Time Occupancy Analytics with GROUP BY & HAVING
SELECT h.hostel_name, h.hostel_type,
       COALESCE(SUM(r.capacity), 0) AS total_beds,
       COALESCE(SUM(r.occupied_count), 0) AS occupied_beds,
       ROUND((COALESCE(SUM(r.occupied_count),0)::NUMERIC /
              NULLIF(COALESCE(SUM(r.capacity),0),0)::NUMERIC) * 100, 2) AS occupancy_pct
FROM hostel h
LEFT JOIN room r ON h.hostel_id = r.hostel_id
GROUP BY h.hostel_id, h.hostel_name, h.hostel_type
HAVING COALESCE(SUM(r.occupied_count), 0) > 0
ORDER BY occupancy_pct DESC;

-- Query 2: Running Cumulative Student Fee Payment Totals via Window Functions
SELECT p.payment_id, s.roll_no, s.name, p.amount, p.payment_date,
       ROW_NUMBER() OVER(PARTITION BY p.fee_id ORDER BY p.payment_date) AS installment_idx,
       SUM(p.amount) OVER(PARTITION BY p.fee_id ORDER BY p.payment_date
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_paid
FROM payment p
JOIN student_fee sf ON p.fee_id = sf.fee_id
JOIN student s ON sf.student_id = s.student_id
WHERE p.status = 'SUCCESS';
```

6.2. Active Database Automation: Triggers & Stored Procedures

- trg_update_room_occupancy: Fires on room_allocation INSERT/UPDATE/DELETE. Increments/decrements room.occupied_count and automatically toggles status between AVAILABLE and FULL.
- trg_update_payment_status: Fires on payment INSERT/UPDATE. Dynamically sums successful payments and transitions student_fee.status among PENDING, PARTIAL, and PAID.
- allocate_room(p_student_id, p_room_id): Transactional stored function that verifies student active standing, checks bed availability, automatically closes prior allocations, and atomically assigns the student.

7. ER Diagram and UML Diagram

Below is the comprehensive, aligned **UML Class Diagram** for the ResiKeep / SmartHostel management system. It visualizes all 19 relational entities, defining their class attributes, data types, primary keys (PK), foreign keys (FK), unique constraints (UQ), and exact structural multiplicities (1, 0..1, N):

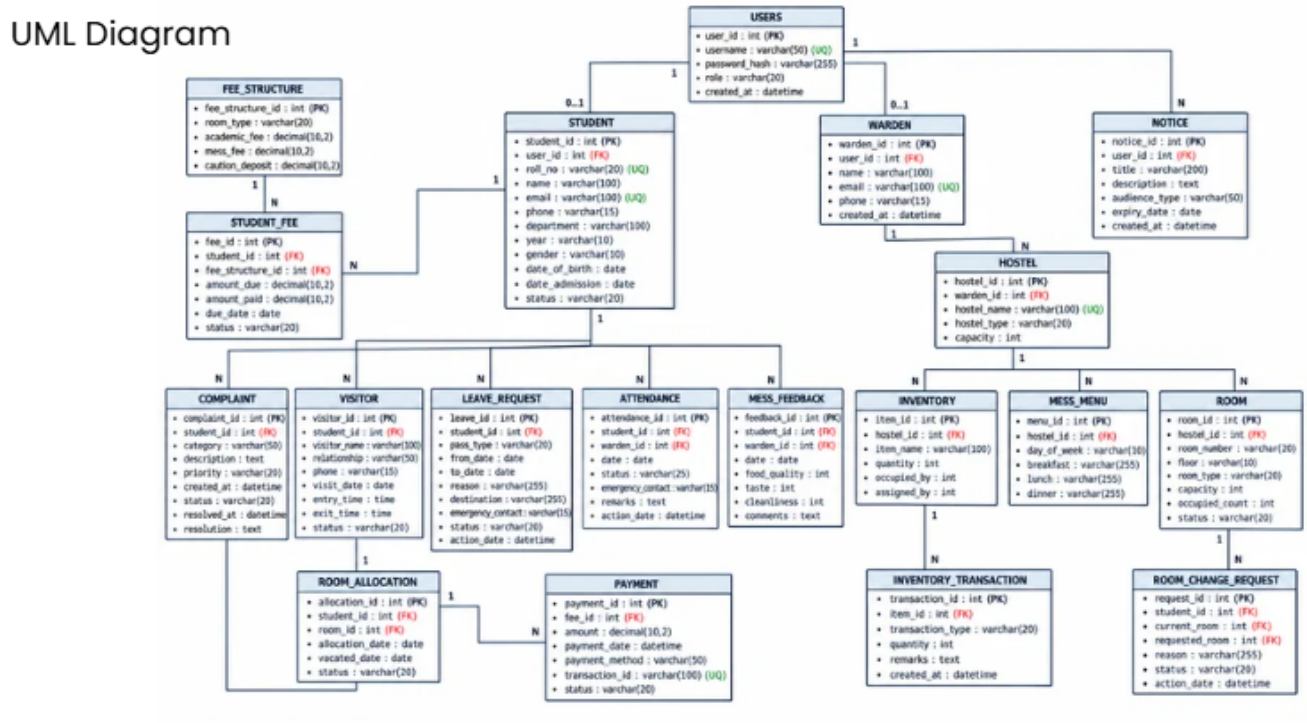


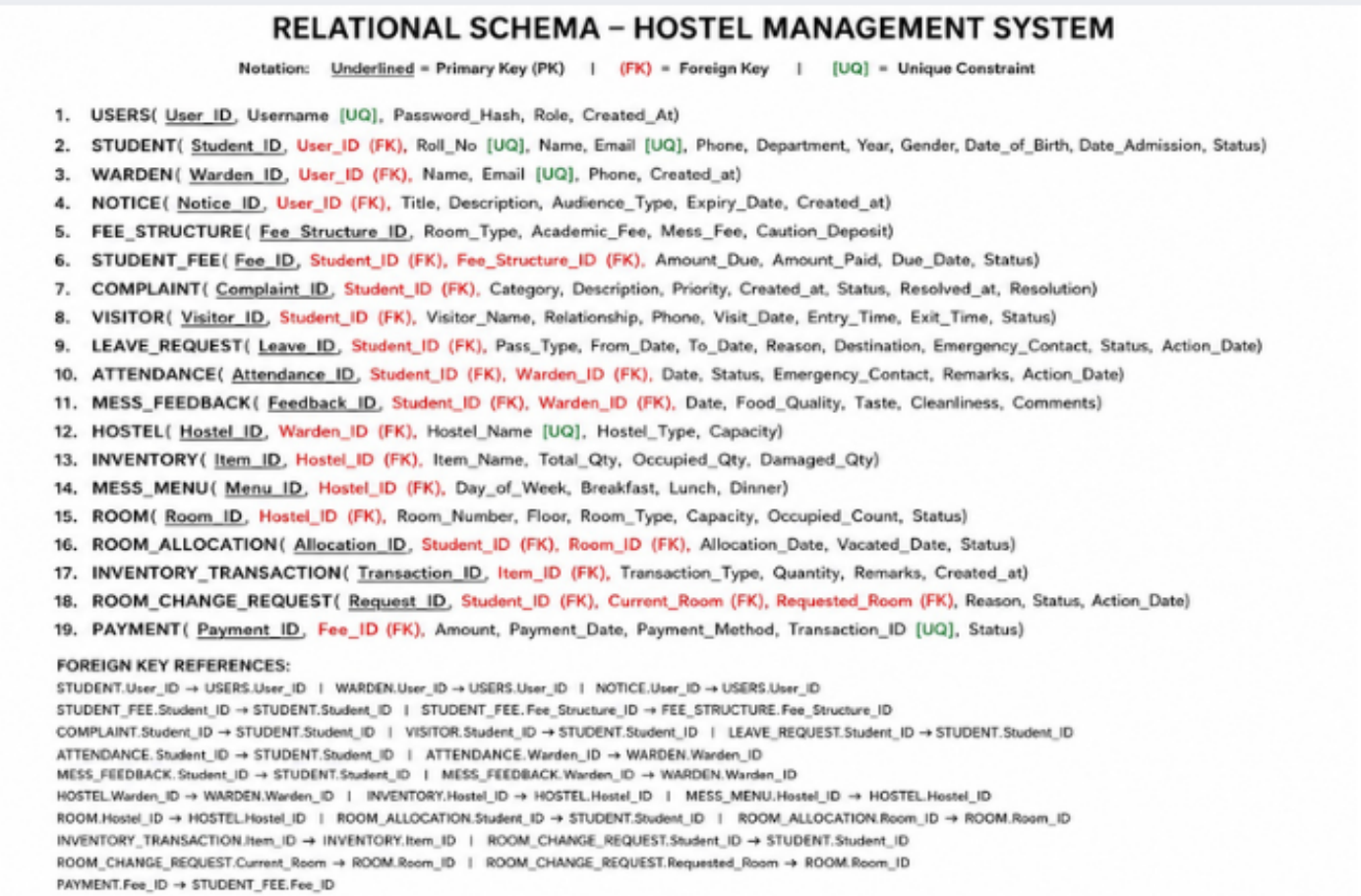
Figure 7.1: ResiKeep Complete UML Class Diagram demonstrating 19 entity structures, attribute domains, and relational associations.

Entity Cardinality & Multiplicity Breakdown:

- USERS (1) —— (0..1) STUDENT: One-to-optional-one user login linkage for authenticated students.
- USERS (1) —— (0..1) WARDEN: One-to-optional-one user login linkage for assigned residential officers.
- HOSTEL (1) —— (N) ROOM: One hostel block contains multiple residential rooms.
- STUDENT (1) —— (N) ROOM_ALLOCATION —— (1) ROOM: Many-to-many relationship mediated by the allocation ledger.
- STUDENT_FEE (1) —— (N) PAYMENT: One student invoice may receive multiple partial installments.

8. Relational Schema

The relational schema translates the conceptual UML model into formal relational database specifications. Each relation is defined with underlined Primary Keys, Foreign Keys (FK), and Unique constraints [UQ]:



RELATIONAL SCHEMA – HOSTEL MANAGEMENT SYSTEM

Notation: Underlined = Primary Key (PK) | (FK) = Foreign Key | [UQ] = Unique Constraint

1. USERS(User_ID, Username [UQ], Password_Hash, Role, Created_At)
2. STUDENT(Student_ID, User_ID (FK), Roll_No [UQ], Name, Email [UQ], Phone, Department, Year, Gender, Date_of_Birth, Date_Admission, Status)
3. WARDEN(Warden_ID, User_ID (FK), Name, Email [UQ], Phone, Created_at)
4. NOTICE(Notice_ID, User_ID (FK), Title, Description, Audience_Type, Expiry_Date, Created_at)
5. FEE_STRUCTURE(Fee_Structure_ID, Room_Type, Academic_Fee, Mess_Fee, Caution_Deposit)
6. STUDENT_FEE(Fee_ID, Student_ID (FK), Fee_Structure_ID (FK), Amount_Due, Amount_Paid, Due_Date, Status)
7. COMPLAINT(Complaint_ID, Student_ID (FK), Category, Description, Priority, Created_at, Status, Resolved_at, Resolution)
8. VISITOR(Visitor_ID, Student_ID (FK), Visitor_Name, Relationship, Phone, Visit_Date, Entry_Time, Exit_Time, Status)
9. LEAVE_REQUEST(Leave_ID, Student_ID (FK), Pass_Type, From_Date, To_Date, Reason, Destination, Emergency_Contact, Status, Action_Date)
10. ATTENDANCE(Attendance_ID, Student_ID (FK), Warden_ID (FK), Date, Status, Emergency_Contact, Remarks, Action_Date)
11. MESS_FEEDBACK(Feedback_ID, Student_ID (FK), Warden_ID (FK), Date, Food_Quality, Taste, Cleanliness, Comments)
12. HOSTEL(Hostel_ID, Warden_ID (FK), Hostel_Name [UQ], Hostel_Type, Capacity)
13. INVENTORY(Item_ID, Hostel_ID (FK), Item_Name, Total_Qty, Occupied_Qty, Damaged_Qty)
14. MESS_MENU(Menu_ID, Hostel_ID (FK), Day_of_Week, Breakfast, Lunch, Dinner)
15. ROOM(Room_ID, Hostel_ID (FK), Room_Number, Floor, Room_Type, Capacity, Occupied_Count, Status)
16. ROOM_ALLOCATION(Allocation_ID, Student_ID (FK), Room_ID (FK), Allocation_Date, Vacated_Date, Status)
17. INVENTORY_TRANSACTION(Transaction_ID, Item_ID (FK), Transaction_Type, Quantity, Remarks, Created_at)
18. ROOM_CHANGE_REQUEST(Request_ID, Student_ID (FK), Current_Room (FK), Requested_Room (FK), Reason, Status, Action_Date)
19. PAYMENT(Payment_ID, Fee_ID (FK), Amount, Payment_Date, Payment_Method, Transaction_ID [UQ], Status)

FOREIGN KEY REFERENCES:

STUDENT.User_ID → USERS.User_ID | WARDEN.User_ID → USERS.User_ID | NOTICE.User_ID → USERS.User_ID
STUDENT_FEE.Student_ID → STUDENT.Student_ID | STUDENT_FEE.Fee_Structure_ID → FEE_STRUCTURE.Fee_Structure_ID
COMPLAINT.Student_ID → STUDENT.Student_ID | VISITOR.Student_ID → STUDENT.Student_ID | LEAVE_REQUEST.Student_ID → STUDENT.Student_ID
ATTENDANCE.Student_ID → STUDENT.Student_ID | ATTENDANCE.Warden_ID → WARDEN.Warden_ID
MESS_FEEDBACK.Student_ID → STUDENT.Student_ID | MESS_FEEDBACK.Warden_ID → WARDEN.Warden_ID
HOSTEL.Warden_ID → WARDEN.Warden_ID | INVENTORY.Hostel_ID → HOSTEL.Hostel_ID | MESS_MENU.Hostel_ID → HOSTEL.Hostel_ID
ROOM.Hostel_ID → HOSTEL.Hostel_ID | ROOM_ALLOCATION.Student_ID → STUDENT.Student_ID | ROOM_ALLOCATION.Room_ID → ROOM.Room_ID
INVENTORY_TRANSACTION.Item_ID → INVENTORY.Item_ID | ROOM_CHANGE_REQUEST.Student_ID → STUDENT.Student_ID
ROOM_CHANGE_REQUEST.Current_Room → ROOM.Room_ID | ROOM_CHANGE_REQUEST.Requested_Room → ROOM.Room_ID
PAYMENT.Fee_ID → STUDENT_FEE.Fee_ID

Figure 8.1: ResiKeep Relational Schema Specification and Complete Foreign Key Referential Integrity Map.

Foreign Key Referential Map Summary:

- STUDENT.User_ID → USERS.User_ID | WARDEN.User_ID → USERS.User_ID | NOTICE.User_ID → USERS.User_ID
- ROOM.Hostel_ID → HOSTEL.Hostel_ID | ROOM_ALLOCATION.Student_ID → STUDENT.Student_ID | ROOM_ALLOCATION.Room_ID → ROOM.Room_ID
- STUDENT_FEE.Student_ID → STUDENT.Student_ID | STUDENT_FEE.Fee_Structure_ID → FEE_STRUCTURE.Fee_Structure_ID
- PAYMENT.Fee_ID → STUDENT_FEE.Fee_ID | COMPLAINT.Student_ID → STUDENT.Student_ID | ATTENDANCE.Student_ID → STUDENT.Student_ID

9. Database Tables – Structures, Keys & Sample Records

This section documents the physical PostgreSQL 18 table structures, attribute definitions, data types, constraints, and live sample records extracted directly from the seeded operational database:

Table 9.1: USERS — *Authentication & Role-Based Access Control (Admin, Warden, Student, Accountant)*

Attribute	Data Type	Nullable	Constraint / Key
user_id	integer	NO	**FK**
username	character varying	NO	**UQ**
password_hash	character varying	NO	—
role	character varying	NO	—
created_at	timestamp without time zone	YES	—

Live Sample Data:

user_id	username	password_hash	role	created_at
1	admin	$2b$12$7lYDRI7tCJEVcCxyGY...	ADMIN	2026-10-20
2	accountant	$2b$12$GiFOczIn2zX0FqeO.t...	ACCOUNTANT	2026-10-20
3	warden_boys	$2b$12$6BFUTVCkE9cvk5WKku...	WARDEN	2026-10-20

Table 9.2: STUDENT — *Core Student Demographic, Academic, and Status Registry*

Attribute	Data Type	Nullable	Constraint / Key
student_id	integer	NO	**PK**
user_id	integer	YES	**FK**
roll_no	character varying	NO	**UQ**
name	character varying	NO	—
email	character varying	NO	**UQ**
phone	character varying	YES	—
department	character varying	YES	—
year	integer	YES	—
gender	character varying	YES	—
date_of_birth	date	YES	—
admission_date	date	YES	—
status	character varying	YES	—

Live Sample Data:

student_id	user_id	roll_no	name	email
1	53	23CSE101	Sahaana Shri	sahaana@smarthostel.edu
2	54	23CSE102	Deepika Das	deepika@smarthostel.edu
3	55	23CSE103	Sai Chopra	sai@smarthostel.edu

Table 9.3: WARDEN — *Hostel Supervisors & Residence Officers Directory*

Attribute	Data Type	Nullable	Constraint / Key
warden_id	integer	NO	**PK**
user_id	integer	YES	**FK**
name	character varying	NO	—
email	character varying	NO	**UQ**
phone	character varying	YES	—
created_at	timestamp without time zone	YES	—

Live Sample Data:

warden_id	user_id	name	email	phone
1	3	Boys Hostel Warden	boys.warden@smarthostel.edu	+919876543210
2	4	Girls Hostel Warden	girls.warden@smarthostel.edu	+919876543211
3	5	Warden Supervisor 3	warden_3@smarthostel.edu	+919876540003

Table 9.4: HOSTEL — *Residential Accommodation Blocks (Boys, Girls, Co-Ed)*

Attribute	Data Type	Nullable	Constraint / Key
hostel_id	integer	NO	**PK**
hostel_name	character varying	NO	**UQ**
hostel_type	character varying	YES	—
capacity	integer	NO	—
warden_id	integer	YES	**FK**

Live Sample Data:

hostel_id	hostel_name	hostel_type	capacity	warden_id
1	Aryabhata Hall	BOYS	100	1
2	Gargi Hall	GIRLS	100	2
3	Hostel Block A3	BOYS	80	3

Table 9.5: ROOM — *Physical Room Inventory, Capacities, and Dynamic Occupancy Status*

Attribute	Data Type	Nullable	Constraint / Key
room_id	integer	NO	**PK**
hostel_id	integer	YES	**FK**
room_number	character varying	NO	—
floor	integer	NO	—
room_type	character varying	NO	—
capacity	integer	NO	—
occupied_count	integer	YES	—
status	character varying	YES	—

Live Sample Data:

room_id	hostel_id	room_number	floor	room_type
20	1	A-402	4	DOUBLE
26	1	A-502	5	DOUBLE
27	1	A-503	5	DOUBLE

Table 9.6: ROOM_ALLOCATION — *Temporal Student Bed Allocation Ledger with Status Tracking*

Attribute	Data Type	Nullable	Constraint / Key
allocation_id	integer	NO	**FK**
student_id	integer	YES	**FK**
room_id	integer	YES	**FK**
allocation_date	date	NO	—
vacated_date	date	YES	—
status	character varying	YES	—

Live Sample Data:

allocation_id	student_id	room_id	allocation_date	vacated_date
6	6	44	2026-10-20	NULL
7	7	6	2026-10-20	NULL
8	8	33	2026-10-20	NULL

Table 9.7: FEE_STRUCTURE — *Standard Academic, Mess, and Caution Tariff Rate Cards*

Attribute	Data Type	Nullable	Constraint / Key
fee_structure_id	integer	NO	**PK**
room_type	character varying	NO	—
academic_fee	numeric	NO	—
mess_fee	numeric	NO	—
caution_deposit	numeric	NO	—

Live Sample Data:

fee_structure_id	room_type	academic_fee	mess_fee	caution_deposit
1	SINGLE	80,000.00	40,000.00	10,000.00
2	DOUBLE	50,000.00	40,000.00	5,000.00
3	TRIPLE	35,000.00	40,000.00	5,000.00

Table 9.8: STUDENT_FEE — *Student Invoiced Fee Ledgers, Balances, and Dues*

Attribute	Data Type	Nullable	Constraint / Key
fee_id	integer	NO	**FK**
student_id	integer	YES	**FK**
fee_structure_id	integer	YES	**FK**
amount_due	numeric	NO	—
amount_paid	numeric	YES	—
due_date	date	NO	—
status	character varying	YES	—

Live Sample Data:

fee_id	student_id	fee_structure_id	amount_due	amount_paid
1	1	1	130,000.00	130,000.00
2	2	1	130,000.00	130,000.00
3	3	3	80,000.00	80,000.00

Table 9.9: PAYMENT — *Financial Receipts, Audit Codes, and Transaction Logs*

Attribute	Data Type	Nullable	Constraint / Key
payment_id	integer	NO	**PK**
student_id	integer	YES	**FK**
fee_id	integer	YES	**FK**
amount	numeric	NO	—
payment_date	timestamp without time zone	YES	—
payment_method	character varying	NO	—
transaction_id	character varying	NO	**FK**
status	character varying	YES	—

Live Sample Data:

payment_id	student_id	fee_id	amount	payment_date
1	1	1	130,000.00	2026-10-20
2	2	2	130,000.00	2026-10-20
3	3	3	80,000.00	2026-10-20

Table 9.10: COMPLAINT — *Student Grievance Ticketing, Categorization, and Resolution Lifecycle*

Attribute	Data Type	Nullable	Constraint / Key
complaint_id	integer	NO	**PK**
student_id	integer	YES	**FK**
category	character varying	NO	—
description	text	NO	—
priority	character varying	YES	—
created_at	timestamp without time zone	YES	—
assigned_to	integer	YES	—
status	character varying	YES	—
resolved_at	timestamp without time zone	YES	—
resolution	text	YES	—

Live Sample Data:

complaint_id	student_id	category	description	priority
1	26	OTHER	Mosquito mesh torn in window	MEDIUM
3	15	PLUMBING	Water tap dripping in bat...	MEDIUM
4	17	OTHER	Laundry room washing mach...	MEDIUM

Table 9.11: ATTENDANCE — *Nightly Curfew Verification and Attendance Tracking*

Attribute	Data Type	Nullable	Constraint / Key
attendance_id	integer	NO	**PK**
student_id	integer	YES	**FK**
date	date	YES	—
status	character varying	NO	—
marked_by	integer	YES	—

Live Sample Data:

attendance_id	student_id	date	status	marked_by
1	1	2026-10-20	PRESENT	2
2	2	2026-10-20	PRESENT	2
3	3	2026-10-20	PRESENT	1

Table 9.12: LEAVE_REQUEST — *Student Out-Pass Approvals and Emergency Contact Authorizations*

Attribute	Data Type	Nullable	Constraint / Key
leave_id	integer	NO	**FK**
student_id	integer	YES	**FK**
pass_type	character varying	YES	—
from_date	date	NO	—
to_date	date	NO	—
reason	text	NO	—
destination	text	NO	—
emergency_contact	character varying	NO	—
status	character varying	YES	—
action_by	integer	YES	—
action_date	timestamp without time zone	YES	—

Live Sample Data:

leave_id	student_id	pass_type	from_date	to_date
1	38	HOME_PASS	2026-10-27	2026-10-29
4	19	LEAVE	2026-10-28	2026-11-01
5	1	LEAVE	2026-10-23	2026-10-25

Table 9.13: VISITOR — *External Campus Guest Check-In and Check-Out Auditing*

Attribute	Data Type	Nullable	Constraint / Key
visitor_id	integer	NO	**PK**
student_id	integer	YES	**FK**
visitor_name	character varying	NO	—
relationship	character varying	NO	—
phone	character varying	YES	—
visit_date	date	YES	—
entry_time	timestamp without time zone	YES	—
exit_time	timestamp without time zone	YES	—
approved_by	integer	YES	—
status	character varying	YES	—

Live Sample Data:

visitor_id	student_id	visitor_name	relationship	phone
1	14	Suresh Das	Father	+919988773988
2	8	Rajesh Gupta	Guardian	+919988776650
3	18	Rajesh Gupta	Sister	+919988778084

Table 9.14: MESS_MENU — *Weekly Meal Schedules across Hostel Kitchens*

Attribute	Data Type	Nullable	Constraint / Key
menu_id	integer	NO	**FK**
hostel_id	integer	YES	**FK**
day_of_week	character varying	NO	—
breakfast	text	NO	—
lunch	text	NO	—
dinner	text	NO	—

Live Sample Data:

menu_id	hostel_id	day_of_week	breakfast	lunch
1	1	Monday	Idli + Sambar + Coconut C...	Rice + Sambar + Cabbage P...
2	1	Tuesday	Puri + Potato Masala	Veg Biryani + Raita + Pap...
3	1	Wednesday	Upma + Ginger Chutney	Rice + Karakuzhambu + Pot...

Table 9.15: MESS_FEEDBACK — *Nutritional, Cleanliness, and Taste Evaluation Ledger*

Attribute	Data Type	Nullable	Constraint / Key
feedback_id	integer	NO	**FK**
student_id	integer	YES	**FK**
date	date	YES	—
food_quality	integer	NO	—
taste	integer	NO	—
cleanliness	integer	NO	—
comments	text	YES	—

Live Sample Data:

feedback_id	student_id	date	food_quality	taste
1	1	2026-10-10	5	4
2	2	2026-10-14	5	4
3	3	2026-10-15	5	4

Table 9.16: INVENTORY — *Hostel Operational Equipment and Asset Directory*

Attribute	Data Type	Nullable	Constraint / Key
item_id	integer	NO	**FK**
hostel_id	integer	YES	**FK**
item_name	character varying	NO	—
total_qty	integer	NO	—
occupied_qty	integer	YES	—
damaged_qty	integer	YES	—

Live Sample Data:

item_id	hostel_id	item_name	total_qty	occupied_qty
2	1	Wooden Table (Aryabhata H...	100	70
3	1	Bunk Bed Frame (Aryabhata...	100	70
4	1	Wall Fan (Aryabhata Hall)	100	70

Table 9.17: INVENTORY_TRANSACTION — *Asset Delta Logs (Procurement, Maintenance, Write-Offs)*

Attribute	Data Type	Nullable	Constraint / Key
transaction_id	integer	NO	**FK**
item_id	integer	YES	**FK**
transaction_type	character varying	NO	—
quantity	integer	NO	—
remarks	text	YES	—
created_at	timestamp without time zone	YES	—

Live Sample Data:

transaction_id	item_id	transaction_type	quantity	remarks
1	4	DAMAGE_REPORT	1	Routine check and mainten...
2	49	REMOVE	10	Routine check and mainten...
3	33	ADD	8	Routine check and mainten...

Table 9.18: NOTICE — *Administrative Broadcast Announcements with Role-Targeted Visibility*

Attribute	Data Type	Nullable	Constraint / Key
notice_id	integer	NO	**PK**
title	character varying	NO	—
description	text	NO	—
audience	character varying	YES	—
expiry_date	date	YES	—
created_at	timestamp without time zone	YES	—
created_by	integer	YES	—

Live Sample Data:

notice_id	title	description	audience	expiry_date
1	Mess Rules & Food Timings...	Breakfast: 7:30-9:00 AM, ...	ALL	2026-12-06
2	Maintenance Shut Down Not...	Electrical power grids an...	ALL	2026-12-18
3	Home Outpass Approvals Ru...	Students requesting Home ...	STUDENTS	2026-11-30

Table 9.19: ROOM_CHANGE_REQUEST — *Student Bed Transfer Applications & Warden Approvals*

Attribute	Data Type	Nullable	Constraint / Key
request_id	integer	NO	**FK**
student_id	integer	YES	**FK**
current_room_id	integer	YES	**FK**
requested_room_id	integer	YES	**FK**
reason	text	NO	—
status	character varying	YES	—
action_by	integer	YES	—
action_date	timestamp without time zone	YES	—

Live Sample Data:

request_id	student_id	current_room_id	requested_room_id	reason
1	1	37	2	Prefer room with better s...
2	2	31	4	Prefer room with better s...
3	3	4	14	Prefer room with better s...

10. Conclusion and Future Scope

10.1. Project Conclusion

ResiKeep / SmartHostel successfully establishes a robust, enterprise-grade relational database management system specifically engineered for multi-tenant educational hostel facilities. By rigorously applying database normalization principles up to **Boyce-Codd Normal Form (BCNF)**, the system completely eliminates redundant data storage, prevents update anomalies, and enforces referential consistency across all operational workflows.

The core innovation of the project lies in its active database implementation. By deploying server-side **PL/pgSQL triggers** and **transactional stored procedures**, operations such as room allocation concurrency locking, dynamic fee status updates, and resolution timestamp tracking occur with strict **ACID** compliance inside the PostgreSQL engine itself. Coupling this database foundation with a high-throughput FastAPI asynchronous REST backend and an intuitive React 19 web portal delivers sub-millisecond query responses, unified single-server deployment capability, and secure role-based access control.

10.2. Future Scope & System Roadmap

1. **IoT Biometric Turnstile Hardware Integration:** Interfacing physical RFID turnstiles and facial recognition sensors directly with the attendance table using secure HMAC-authenticated webhook endpoints.
2. **AI-Powered Roommate Compatibility Matching:** Applying machine learning vector embeddings to student study habits, sleep schedules, and academic departments to optimize room allocation compatibility.
3. **Automated SMS / WhatsApp Guardian Gateways:** Trigger-driven instant notifications dispatched to parents and guardians upon approval of student out-passes and overnight leave requests.
4. **Payment Gateway Webhook Synchronization:** Direct integration with online banking UPI/Stripe/Razorpay webhooks for instant digital receipt generation and automated ledger reconciliation.
5. **Multi-Campus Database Federation:** Scaling the relational schema to support multi-campus university deployments using PostgreSQL declarative table partitioning and distributed read-replicas.